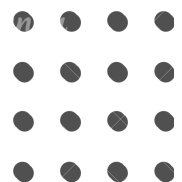




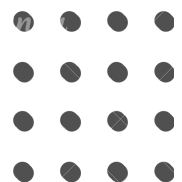
VMES Grinds





Pilot Program - Week 2 - Day 2

Hashing





3. Longest Substring Without Repeating Characters





3. Longest Substring Without Repeating Characters

Medium

Topics

Companies

Hint

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

Input: `s = "abcabcbb"`

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: `s = "bbbbbb"`

Output: 1

Explanation: The answer is "b", with the length of 1.



Example 3:

Input: `s = "pwwkew"`

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- `0 <= s.length <= 5 * 104`
- `s` consists of English letters, digits, symbols and spaces.



Explanation





Thought Process

- **Linear iteration through the string**
- **Use sliding window**
- **Check existing character**
- **Compare and update length**



Example: s = "abcabcbb"

[] → [a] → [ab] → [abc] = 3

[abca] → [bca] = 3

Λ Λ

[bcab] → [cab] = 3

Λ Λ

[cabc] → [abc] = 3

Λ Λ

[abcb] → [cb] = 2

Λ Λ

[cbb] → [b] = 1 → []

ΛΛ

Example: s = "pwwkew"

[] → [p] → [pw] = 2

[pww] → [w] = 1

ΛΛ

[wk] → [wke] = 3

[wkew] → [kew] = 3

Λ Λ

[kew] = 3 → [ew] = 2

[ew] = 2 → [w] = 1

[w] = 1 → []



Code





```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        |
        track = {}
        subStr = ''
        long = 0
        if len(s) < 2:
            return len(s)
        for i in range(len(s)):
            subStr = subStr + s[i]
            if s[i] in track and s[i] in subStr[:-1]:
                subStr = s[track[s[i]]+1:i+1]
            track[s[i]] = i
            if long < len(subStr):
                long = len(subStr)
        return long
```



Optimized Code





```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        seen = {}
        left = 0
        res = 0

        for right, ch in enumerate(s):
            if ch in seen and seen[ch] >= left:
                left = seen[ch] + 1

            seen[ch] = right
            res = max(res, right - left + 1)

        return res
```



Time and Space Complexity Analysis





```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        seen = {}
        left = 0
        res = 0

        for right, ch in enumerate(s): ← Time - O(n)
            if ch in seen and seen[ch] >= left:
                left = seen[ch] + 1

            seen[ch] = right ← Space - O(n)
            res = max(res, right - left + 1)

        return res
```



Key Takeaway





- **Avoid substring operations**
 - **Strings are immutable**
 - **Mostly $\rightarrow O(n^2)$**
- **Use a sliding window with pointers**
- **Use a hash map**
 - **Constant-Time Lookup**
 - **$O(1)$ duplicate detection**



Thank You !

